LAB REPORT- 03

Department of Computer Science & Engineering (CSE)
Faculty of Science and Information Technology (FSIT)
Spring 2026

Course Title	**:** Operating Systems Lab
Course Code	**:** CSE324
Course Credit	**:** 1.5
Date of Submission	**: April 09 , 2026**
Topic Covered	**: Implementation of Banker's Algorithm**

Submitted By

Shafin Ahmed
232-15-184
65_A1
Department of Computer Science & Engineering

Submitted To

Husne Mubarak
Lecturer
Department of Computer Science & Engineering
Faculty of FSIT

Comment By Course Teacher:

Table of Contents

Experiment No: 01

Experiment Name: Implementation of Banker's Algorithm (Deadlock Avoidance)

Program Name: ba.c

Introduction

Overview

In this experiment, I have implemented the Banker's Algorithm using a C program to understand deadlock avoidance in operating systems. The primary objective of this experiment has been to determine whether a system is in a safe state or not and to find a safe sequence of process execution.

The Banker's Algorithm ensures that resource allocation is done in such a way that the system never enters an unsafe state.

Topics Covered

- Banker's Algorithm
- Deadlock Avoidance
- Safe State and Unsafe State
- Need Matrix Calculation
- Resource Allocation
- Safe Sequence Determination

Environment / Platform

I have performed this experiment using:

- Operating System: Ubuntu Linux

- Programming Language: C
- Editor: Nano
- Compiler: GCC
- Terminal Commands

Execution Steps

nano ba.c

gcc ba.c -o ba

./ba

Task

Description

The task was to implement the Banker's Algorithm that:

- Takes input:
  - Number of processes
  - Number of resources
  - Total instances of each resource
  - Max matrix
  - Allocation matrix
- Calculates:
  - Need matrix
  - Available resources
- Determines:
  - Whether the system is in a safe state
  - Safe sequence of processes (if exists)

Source Code

```
// Shafin Ahmed_232-15-184_65-A1

//Banker's Algorithm

#include<stdio.h>

int main()

{
```

```
    int n,m;
    printf("enter total number of processes: ");
    scanf("%d",&n);
    printf("enter total number of resources: ");
    scanf("%d",&m);
//array related to resource
    int totalres[m]; //r1,r2,r3
    int available[m]; // currently available resources
    int work[m]; // Temporary array for checking safety
//tempo available resource track korar jonne
//process related matrices
    int max[n][m]; // maximum matrics
    int allocation[n][m]; // allocation matrics
    int need[n][m]; // max - allocation
    int safeseq[n]; //safe seq store korbe
    int finish[n]; //finish process track kore ...
//0= finish na, 1=finish
    int i,j,totalallocated;
    int count=0; //total num of finish process

//finish array init
    for(i=0; i<n; i++)
        finish[i]=0;
//input total resources
    printf("enter total instances of each resource:\n");
    for(i=0; i<m; i++)
```

```
    {
        printf("resource R%d: ",i+1);
        scanf("%d",&totalres[i]);
    }
//max matrix input
    printf("enter Max matrix:\n");
    for(i = 0; i < n; i++)
    {
        for(j = 0; j < m; j++)
        {
            printf("Max for P%d, R%d: ", i, j + 1);
            scanf("%d", &max[i][j]);
        }
    }
// allocation matrix input
    printf("enter Allocation matrix:\n");
    for(i = 0; i < n; i++)
    {
        for(j = 0; j < m; j++)
        {
            printf("Allocation for P%d, R%d: ", i, j + 1);
            scanf("%d", &allocation[i][j]);
        }
    }
//need matrix calculation
    for(i=0; i<n; i++)
```

```
    {
        for(j=0; j<m; j++)
        {
            need[i][j]=max[i][j]-allocation[i][j];
        }
    }
//calculating available
    for(i=0; i<m; i++)
    {
        totalallocated = 0;
//sum of all allocation for resource i
        for(j=0; j<n; j++)
        {
            totalallocated += allocation[j][i];
        }
        available[i] = totalres[i] - totalallocated;
// work initially same as available
        work[i] = available[i];
    }
//ekhon khali print hobe matrix gula
    printf("Allocation matrix:\n");
    for(i = 0; i < n; i++)
    {
        for(j = 0; j < m; j++)
        {
            printf("%d ", allocation[i][j]);
```

```
    }
    printf("\n");
}
printf("Max matrix:\n");
for(i = 0; i < n; i++)
{
    for(j = 0; j < m; j++)
    {
        printf("%d ", max[i][j]);
    }
    printf("\n");
}

printf("Need matrix:\n");
for(i = 0; i < n; i++)
{
    for(j = 0; j < m; j++)
    {
        printf("%d ", need[i][j]);
    }
    printf("\n");
}
printf("Available resources: ");
for(i = 0; i < m; i++)
{
    printf("%d ", available[i]);
```

```
    }
    printf("\n");
    //Bankers Algorithm for Safety Check and the sequences
    while(count < n)
    {
        int found=0; //ei loop e kono process execute hoise kina check korbe
        for(i=0; i<n; i++)
        {
//not finish process check kore
            if(finish[i]==0)
            {
                int canexecute = 1;
//check need<=work
                for(j=0; j<m; j++)
                {
                    if(need[i][j]>work[j])
                    {
                        canexecute=0; //can't execute
                        break;
                    }
                }
//if process execute
                if(canexecute)
                {
//work er sathe allocated jog
                    for(j=0; j<m; j++)
```

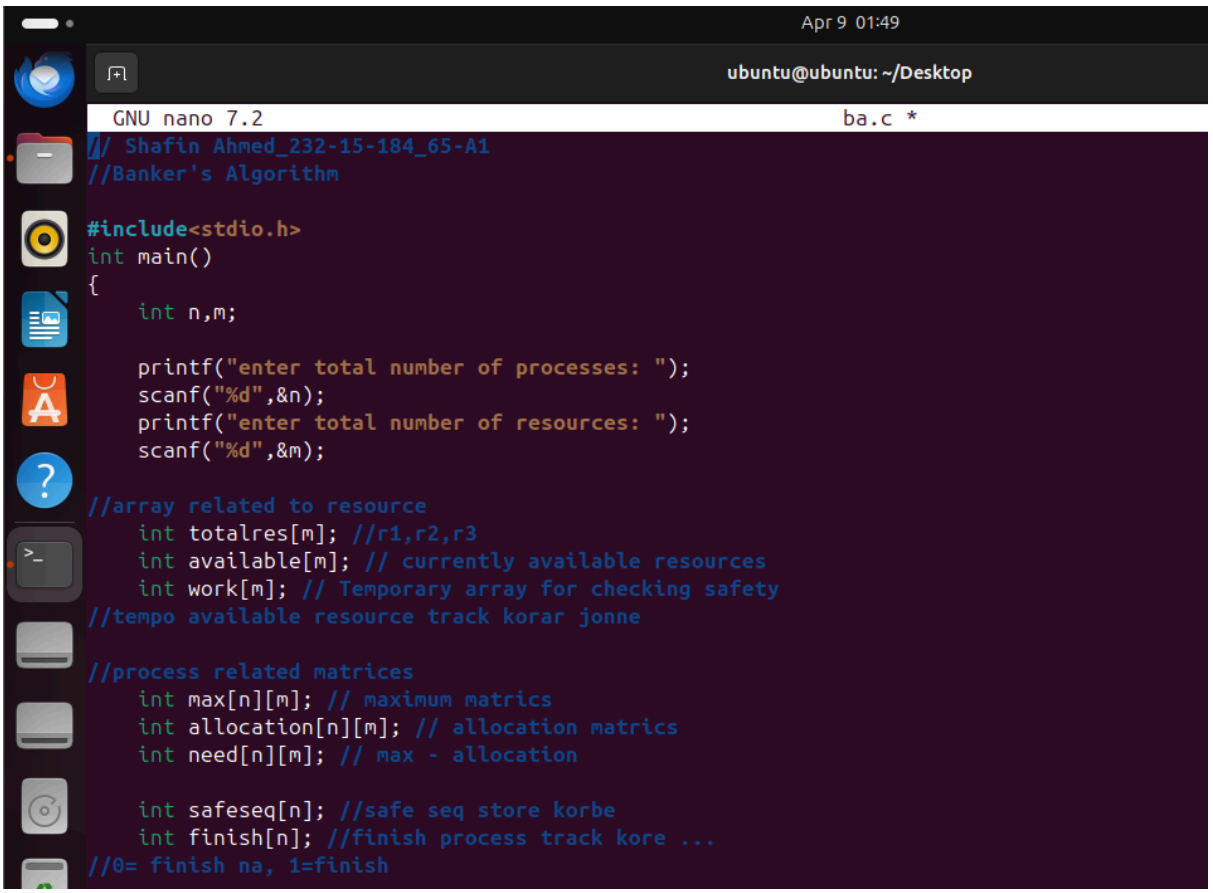
```
                {
                    work[j] += allocation[i][j];
                }
//process finish er marking
                safeseq[count++]=i;
                finish[i]=1;
                found = 1; // atleast ekta process execute hoilo
            }
        }
    }
//if no process execute than unsafe state
    if(found==0)
    {
        printf("System is not in safe state!\n");
        return 0;
    }
  }
//print Safe sequence
  printf("System is in Safe state.\nSafe sequence: ");
  for(i=0; i<n; i++)
  {
    printf("P%d ",safeseq[i]);
  }
  return 0;
}
```

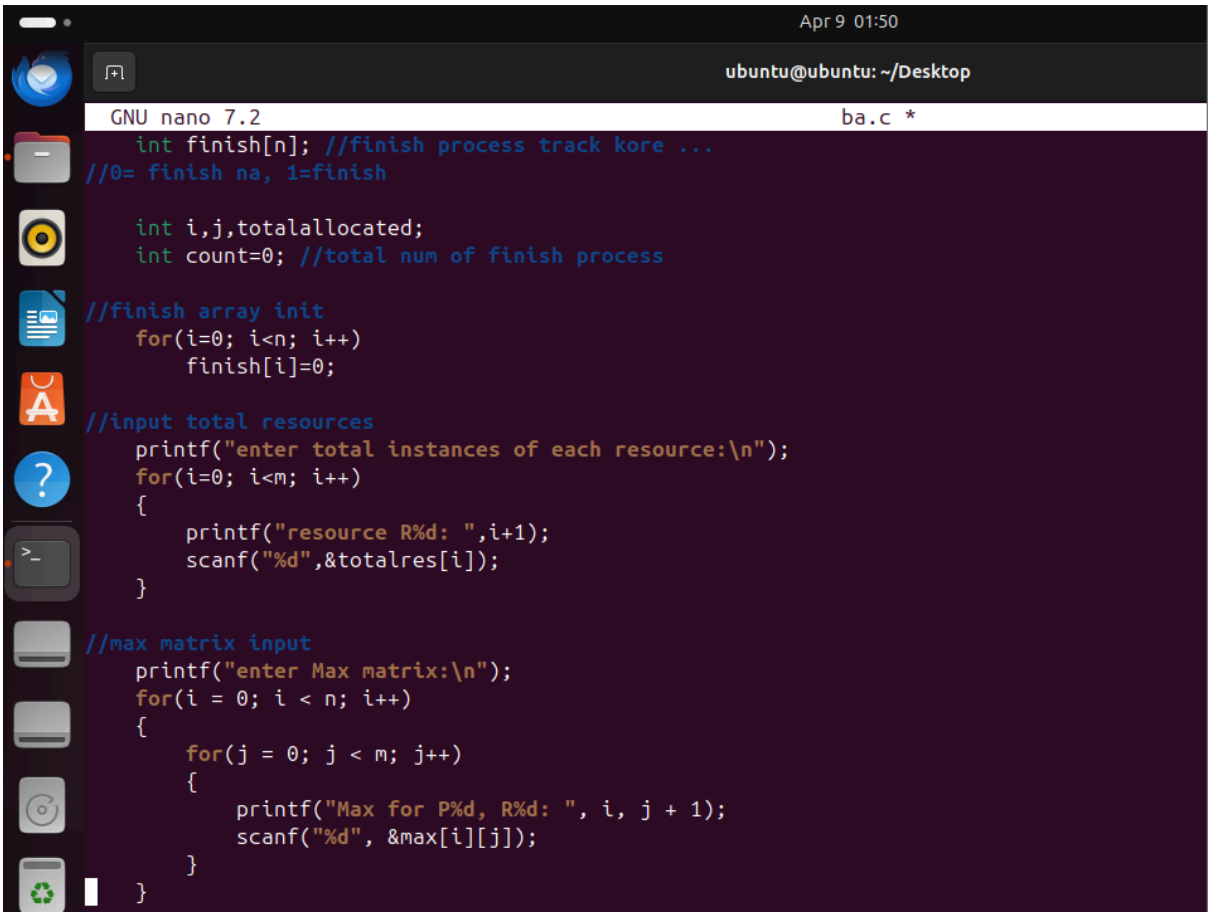
Code Description

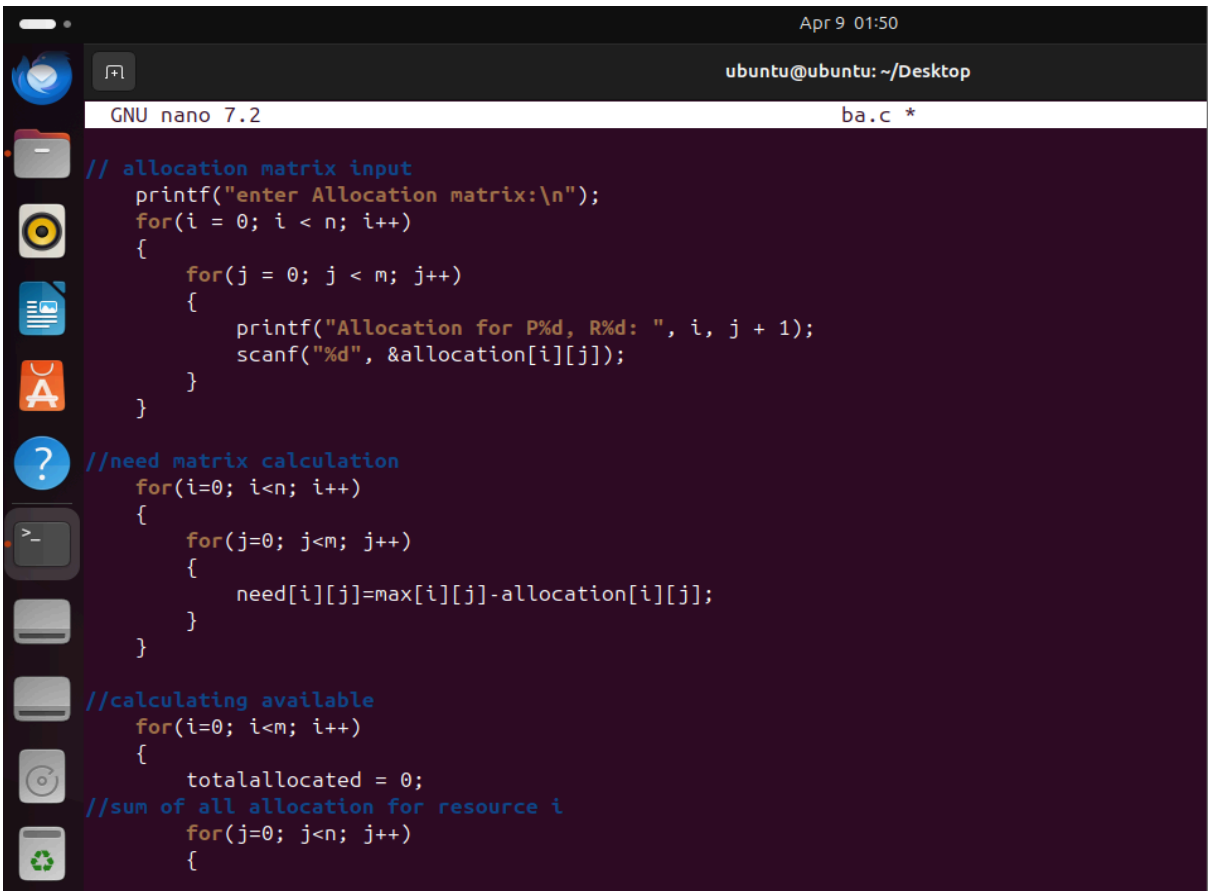
- I have taken the number of processes (`n`) and resources (`m`) as input.
- I have declared the following arrays:
  - `totalres[]` → stores total resources
  - `available[]` → currently available resources
  - `work[]` → temporary array for safety checking
  - `max[][]` → maximum demand of each process
  - `allocation[][]` → allocated resources
  - `need[][]` → remaining resource need
  - `finish[]` → tracks completed processes
  - `safeseq[]` → stores safe sequence
- I have initialized all processes as unfinished (`finish[i] = 0`).
- I have taken input for:
  - Total resources
  - Max matrix
  - Allocation matrix
- I have calculated the **Need Matrix** using:
  - `Need = Max - Allocation`
- I have calculated **Available Resources** by subtracting allocated resources from total resources.
- I have used the Banker's Algorithm logic:
  - I checked if `Need <= Work`
  - If true, the process executes
  - Allocated resources are added back to `Work`
  - Process is marked as finished
- I repeated this process until:
  - All processes are finished (Safe State)
  - Or no process can execute (Unsafe State)
- Finally, I printed:
  - Allocation, Max, Need matrices
  - Available resources
  - Safe sequence (if exists)

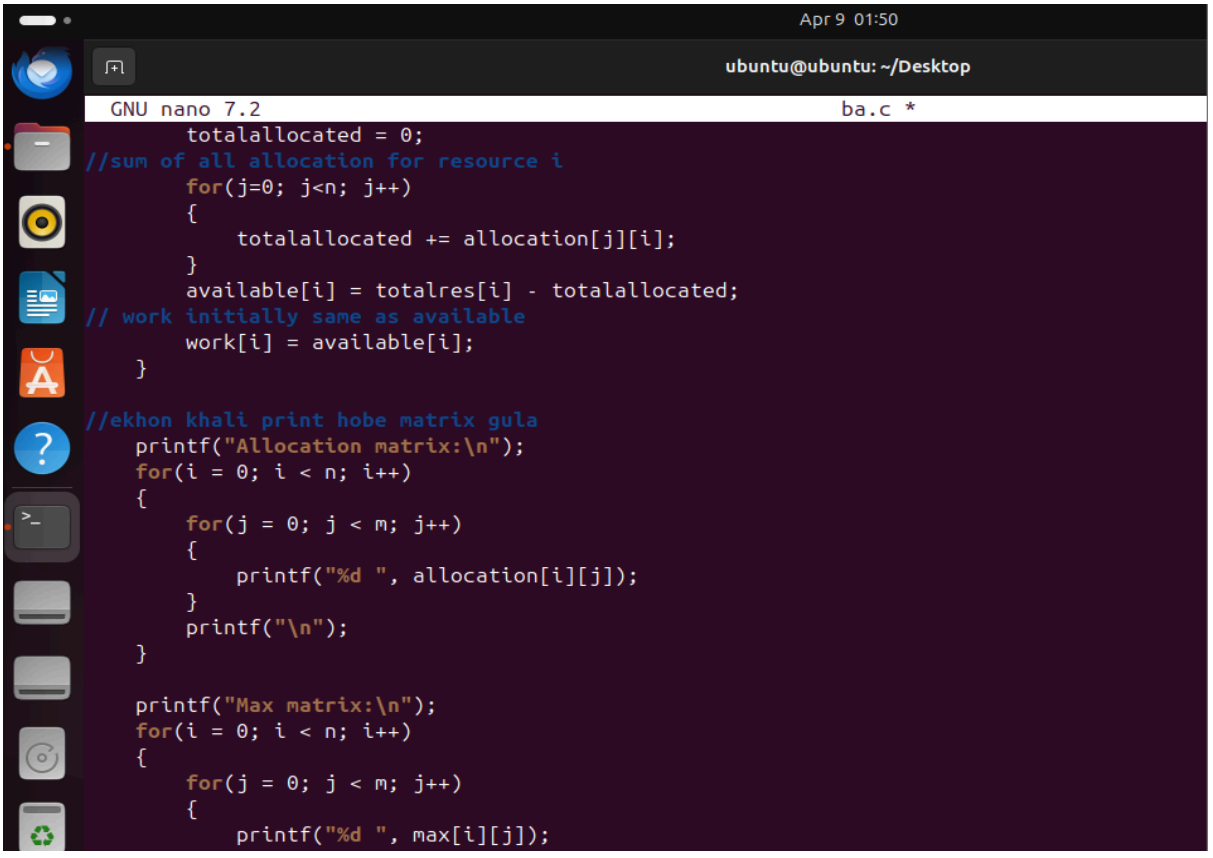
Input & Output

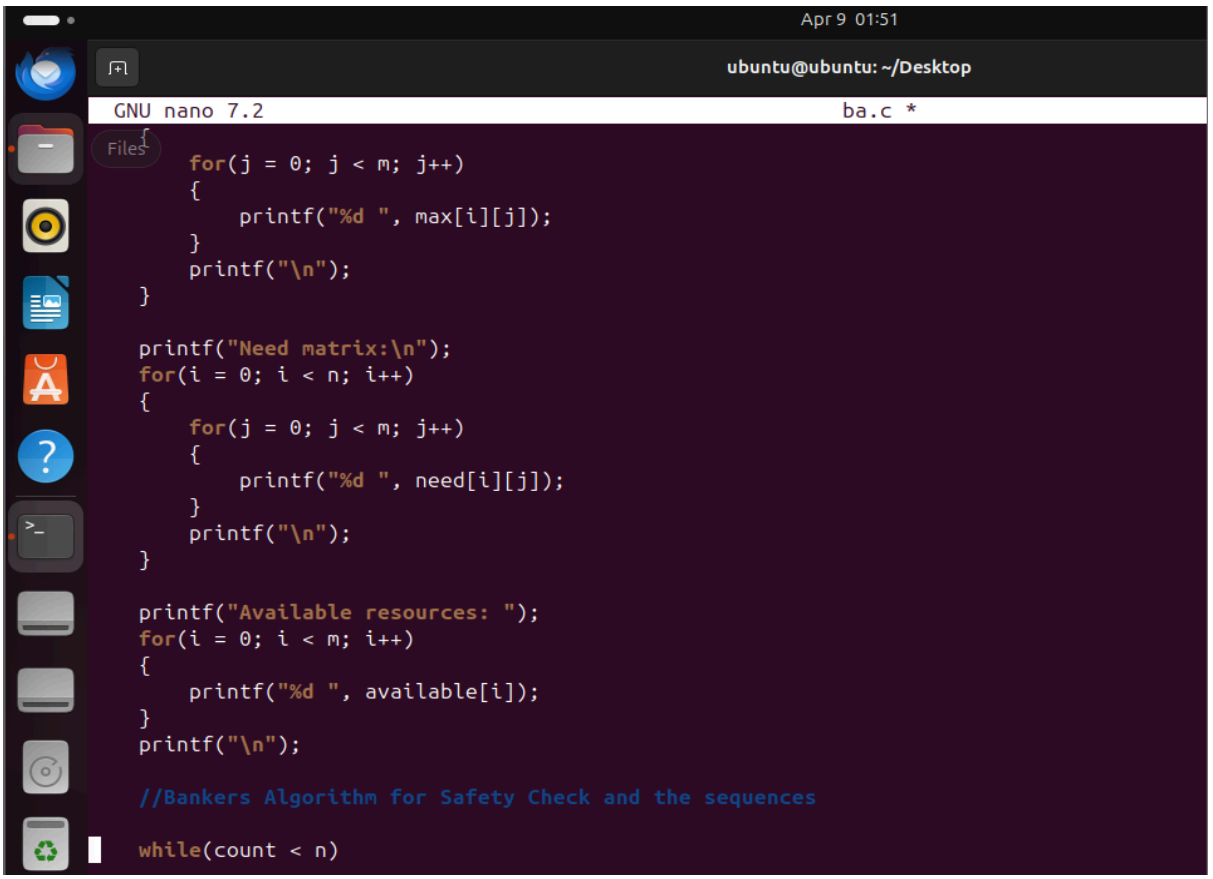
Figure 1: Code Written in Nano Editor

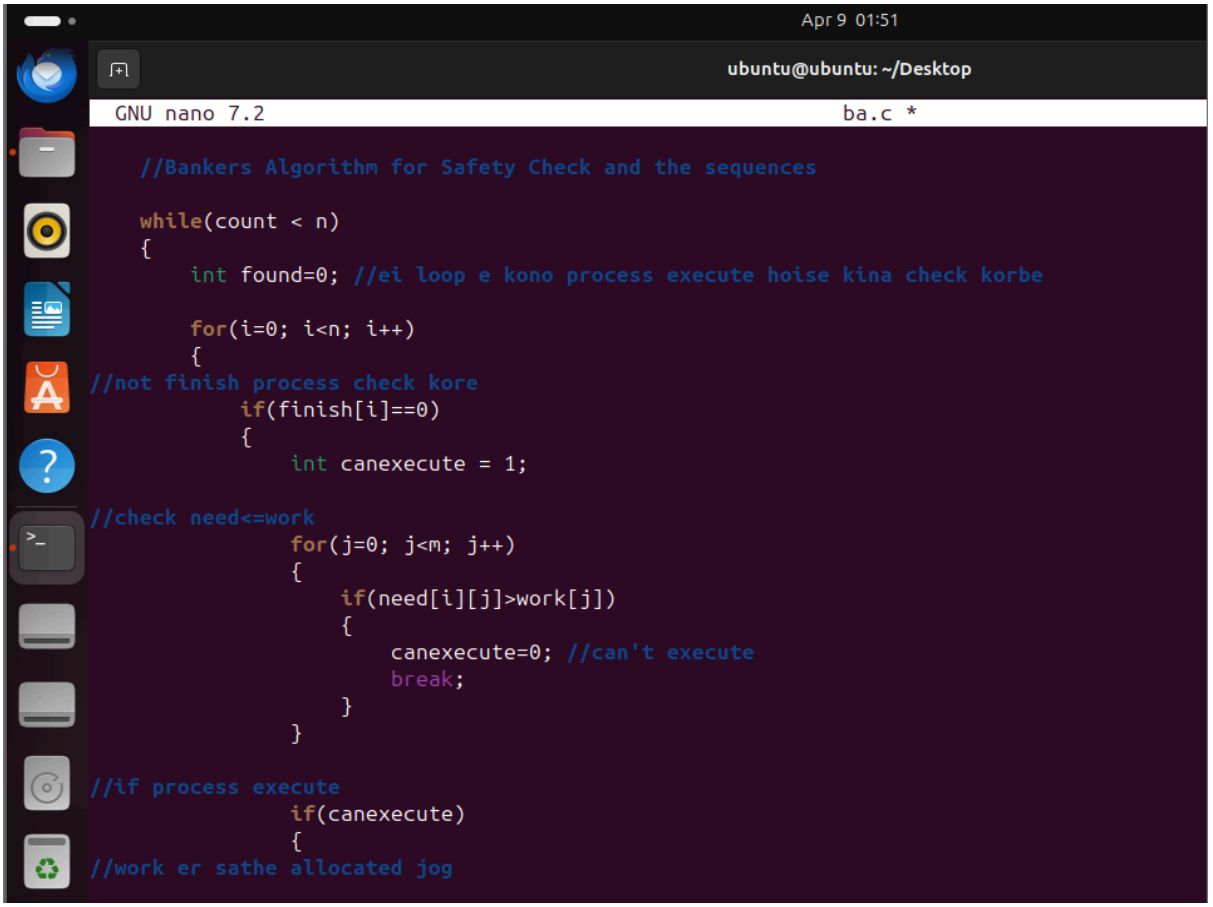

Apr 9 01:49
ubuntu@ubuntu: ~/Desktop
GNU nano 7.2 ba.c *
// Shafin Ahmed_232-15-184_65-A1
//Banker's Algorithm
#include<stdio.h>
int main()
{
int n,m;
printf("enter total number of processes: ");
scanf("%d",&n);
printf("enter total number of resources: ");
scanf("%d",&m);
//array related to resource
int totalres[m]; //r1,r2,r3
int available[m]; // currently available resources
int work[m]; // Temporary array for checking safety
//tempo available resource track korar jonne
//process related matrices
int max[n][m]; // maximum matrics
int allocation[n][m]; // allocation matrics
int need[n][m]; // max - allocation
int safeseq[n]; //safe seq store korbe
int finish[n]; //finish process track kore ...
//0= finish na, 1=finish


Apr 9 01:50
ubuntu@ubuntu: ~/Desktop
GNU nano 7.2 ba.c *
int finish[n]; //finish process track kore ...
//0= finish na, 1=finish
int i,j,totalallocated;
int count=0; //total num of finish process
//finish array init
for(i=0; i<n; i++)
finish[i]=0;
//input total resources
printf("enter total instances of each resource:\n");
for(i=0; i<m; i++)
{
printf("resource R%d: ",i+1);
scanf("%d",&totalres[i]);
}
//max matrix input
printf("enter Max matrix:\n");
for(i = 0; i < n; i++)
{
for(j = 0; j < m; j++)
{
printf("Max for P%d, R%d: ", i, j + 1);
scanf("%d", &max[i][j]);
}
}


Apr 9 01:50
ubuntu@ubuntu: ~/Desktop
GNU nano 7.2 ba.c *
// allocation matrix input
printf("enter Allocation matrix:\n");
for(i = 0; i < n; i++)
{
for(j = 0; j < m; j++)
{
printf("Allocation for P%d, R%d: ", i, j + 1);
scanf("%d", &allocation[i][j]);
}
}
//need matrix calculation
for(i=0; i<n; i++)
{
for(j=0; j<m; j++)
{
need[i][j]=max[i][j]-allocation[i][j];
}
}
//calculating available
for(i=0; i<m; i++)
{
totalallocated = 0;
//sum of all allocation for resource i
for(j=0; j<n; j++)
{


Apr 9 01:50
ubuntu@ubuntu: ~/Desktop
GNU nano 7.2 ba.c *
totalallocated = 0;
//sum of all allocation for resource i
for(j=0; j<n; j++)
{
totalallocated += allocation[j][i];
}
available[i] = totalres[i] - totalallocated;
// work initially same as available
work[i] = available[i];
}
//ekhon khali print hobe matrix gula
printf("Allocation matrix:\n");
for(i = 0; i < n; i++)
{
for(j = 0; j < m; j++)
{
printf("%d ", allocation[i][j]);
}
printf("\n");
}
printf("Max matrix:\n");
for(i = 0; i < n; i++)
{
for(j = 0; j < m; j++)
{
printf("%d ", max[i][j]);


Apr 9 01:51
ubuntu@ubuntu: ~/Desktop
GNU nano 7.2
ba.c *
Files
for(j = 0; j < m; j++)
{
printf("%d ", max[i][j]);
}
printf("\n");
}
printf("Need matrix:\n");
for(i = 0; i < n; i++)
{
for(j = 0; j < m; j++)
{
printf("%d ", need[i][j]);
}
printf("\n");
}
printf("Available resources: ");
for(i = 0; i < m; i++)
{
printf("%d ", available[i]);
}
printf("\n");
//Bankers Algorithm for Safety Check and the sequences
while(count < n)


Apr 9 01:51
ubuntu@ubuntu: ~/Desktop
GNU nano 7.2
ba.c *
//Bankers Algorithm for Safety Check and the sequences
while(count < n)
{
int found=0; //ei loop e kono process execute hoise kina check korbe
for(i=0; i<n; i++)
{
//not finish process check kore
if(finish[i]==0)
{
int canexecute = 1;
//check need<=work
for(j=0; j<m; j++)
{
if(need[i][j]>work[j])
{
canexecute=0; //can't execute
break;
}
}
//if process execute
if(canexecute)
{
//work er sathe allocated jog

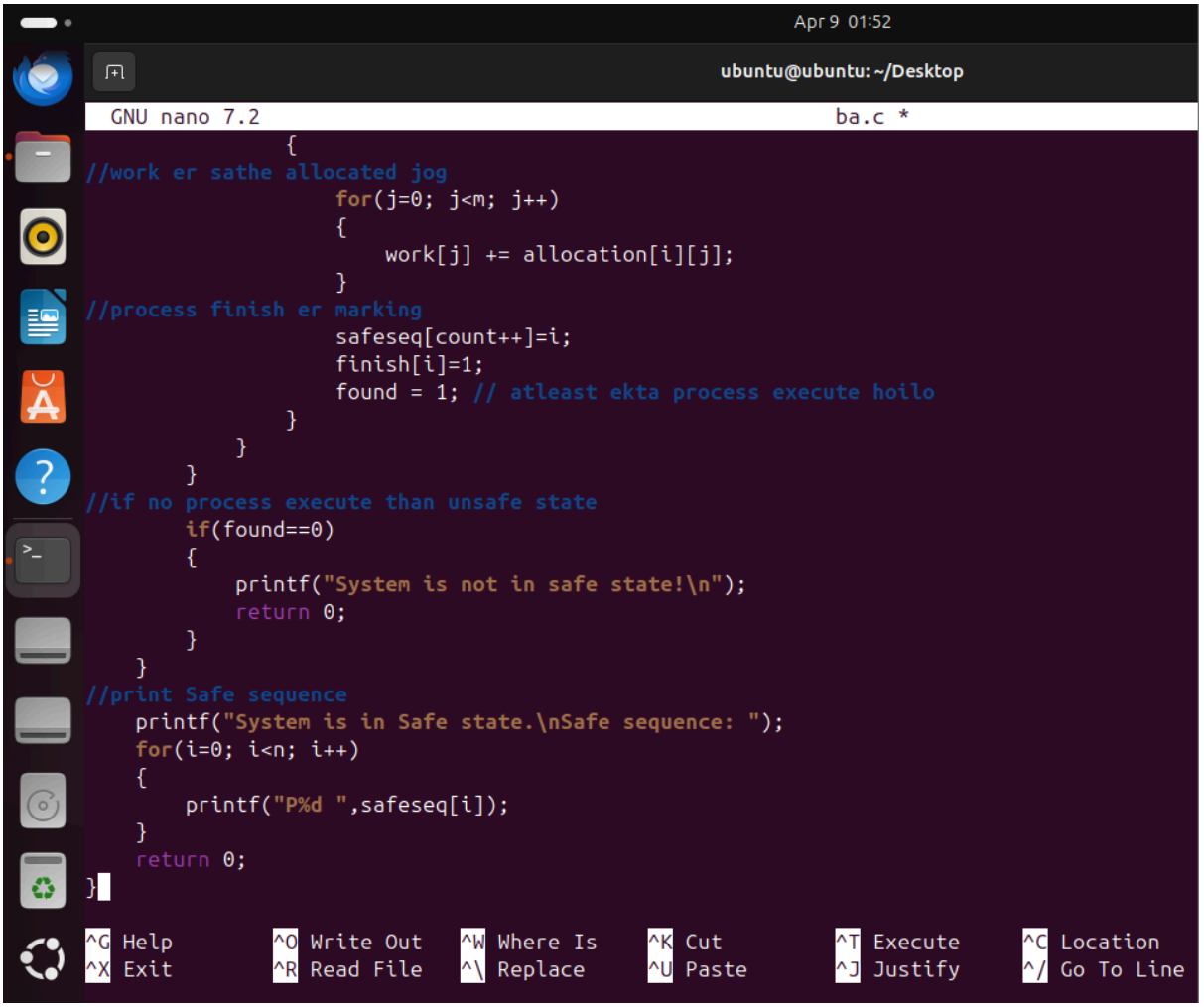


Figure 2: Compilation in Terminal (GCC)

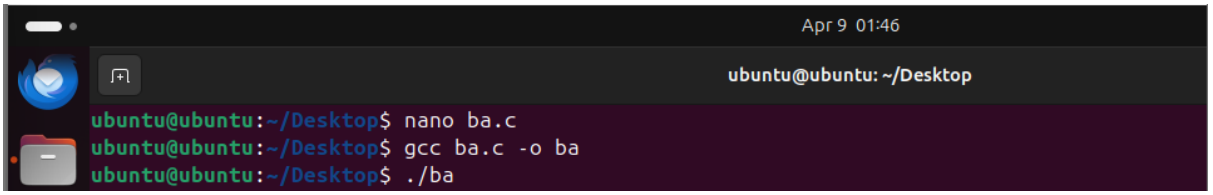
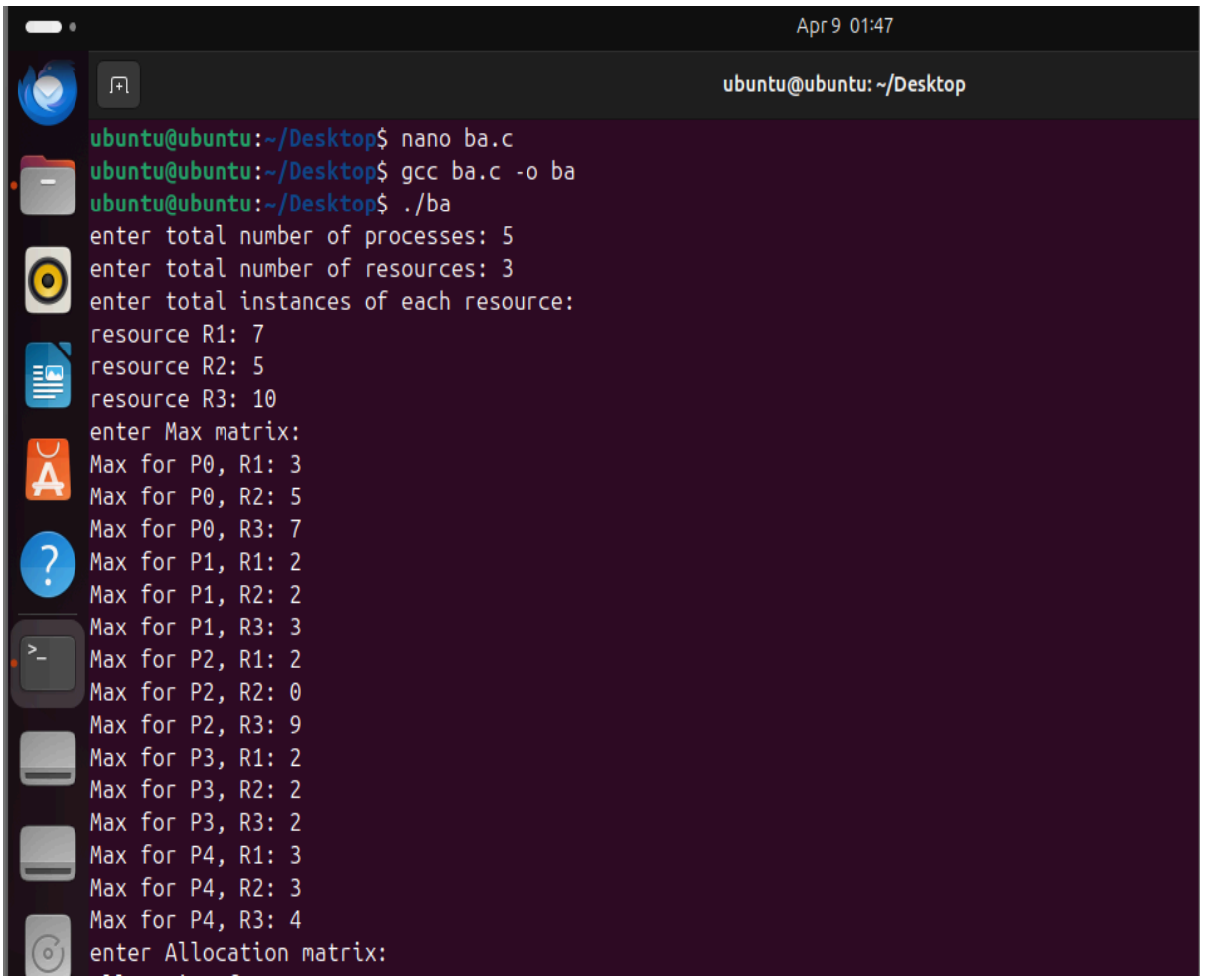
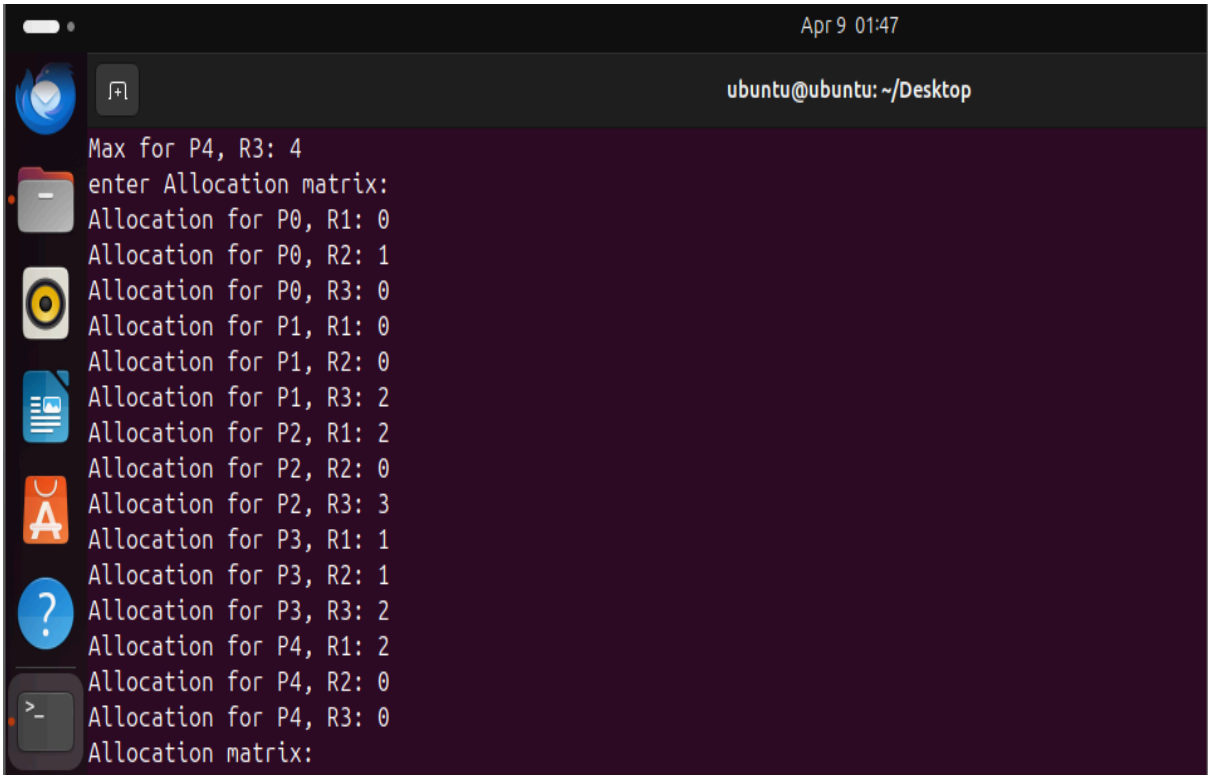
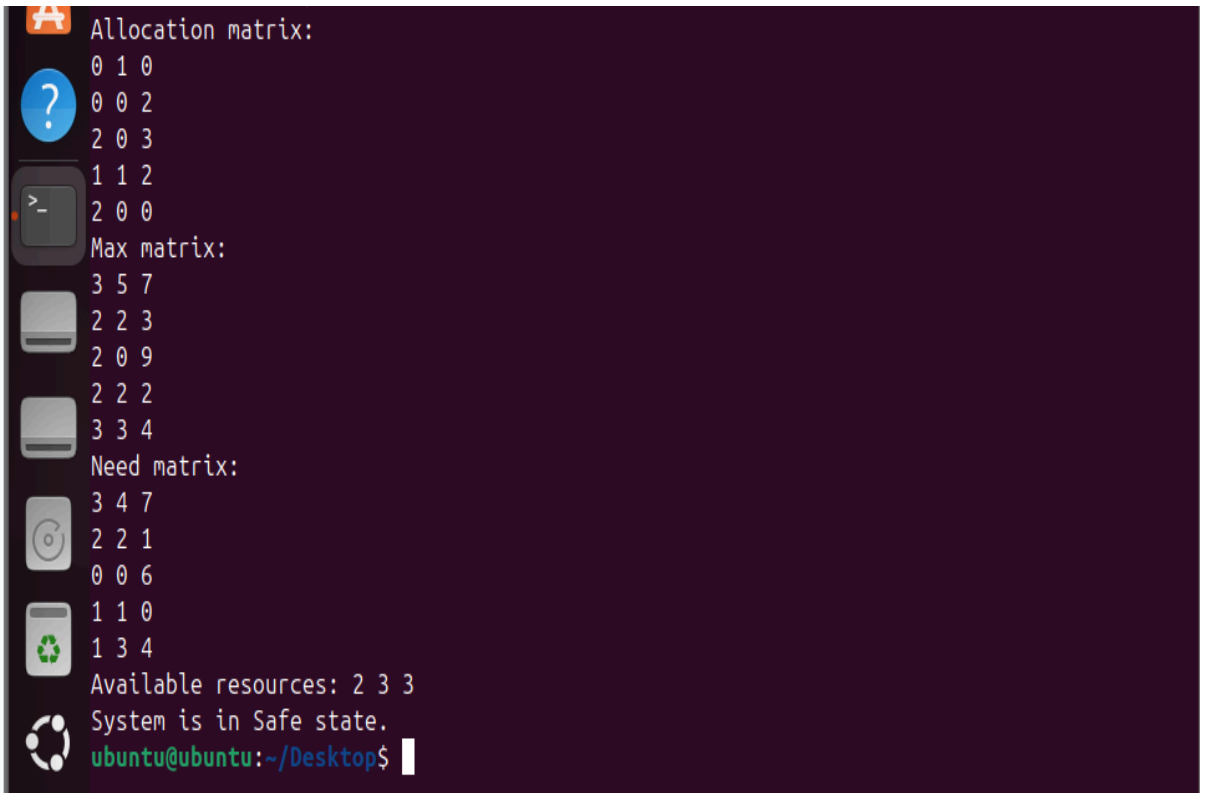


Figure 3: Program Execution Input & Output


Apr 9 01:47
ubuntu@ubuntu: ~/Desktop
ubuntu@ubuntu:~/Desktop$ nano ba.c
ubuntu@ubuntu:~/Desktop$ gcc ba.c -o ba
ubuntu@ubuntu:~/Desktop$ ./ba
enter total number of processes: 5
enter total number of resources: 3
enter total instances of each resource:
resource R1: 7
resource R2: 5
resource R3: 10
enter Max matrix:
Max for P0, R1: 3
Max for P0, R2: 5
Max for P0, R3: 7
Max for P1, R1: 2
Max for P1, R2: 2
Max for P1, R3: 3
Max for P2, R1: 2
Max for P2, R2: 0
Max for P2, R3: 9
Max for P3, R1: 2
Max for P3, R2: 2
Max for P3, R3: 2
Max for P4, R1: 3
Max for P4, R2: 3
Max for P4, R3: 4
enter Allocation matrix:


Apr 9 01:47
ubuntu@ubuntu: ~/Desktop
Max for P4, R3: 4
enter Allocation matrix:
Allocation for P0, R1: 0
Allocation for P0, R2: 1
Allocation for P0, R3: 0
Allocation for P1, R1: 0
Allocation for P1, R2: 0
Allocation for P1, R3: 2
Allocation for P2, R1: 2
Allocation for P2, R2: 0
Allocation for P2, R3: 3
Allocation for P3, R1: 1
Allocation for P3, R2: 1
Allocation for P3, R3: 2
Allocation for P4, R1: 2
Allocation for P4, R2: 0
Allocation for P4, R3: 0
Allocation matrix:



Discussion

Reflection

Through this experiment, I have learned how the Banker's Algorithm prevents deadlock by ensuring the system remains in a safe state. I have understood how resource allocation decisions are made dynamically based on available resources and process requirements.

I have also learned how to compute the Need matrix and how safe sequences are generated.

Challenges Faced

Initially, I had to carefully manage multiple matrices and ensure correct calculations for Need and Available resources. I also ensured proper implementation of the safety algorithm logic. After resolving these issues, the program executed successfully.

Conclusion

In this experiment, I have successfully implemented the Banker’s Algorithm using C in a Linux environment. I have verified system safety and determined safe execution sequences.

This experiment has helped me understand one of the most important deadlock avoidance techniques used in operating systems.
